Description | Resources

Window functions apply aggregate and ranking functions over a particular window (set of rows). OVER clause is used with window functions to define that window. OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS new_column
FROM table_name;
```

Questions 1/9

1. Create order_rank for all the orders based on the order data. Also, observe if more than one order_Id have gotten same rank or not.

   Expected Output: Order_id, order_date and order_rank.

Query

```
select
 order_id,
 order_date,
 rank()over(order by order_date asc) as order_rank
from orders
order by 2 asc
```

Reset | Run query | Submit query

Database | Result

order_id	order_date	order_rank
10248	1996-07-04	1
10249	1996-07-05	2
10250	1996-07-08	3
10251	1996-07-08	3
10252	1996-07-09	5

Description | Resources

(set of rows). OVER clause is used with window functions to define that window. OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS new_column
FROM table_name;
```

Questions 2/9

2. Create shipping_rank for all the orders based on the orders data. The first shipped order should get the first rank. Also, observe if more than one order_Id have gotten the same rank or not.

   Expected Output: Order_id, shipped_date and shipping_rank.

Query

```
select
 order_id,
 shipped_date,
 rank() over(order by shipped_date asc ) as shipping_rank
from orders
order by 2 asc
```

Reset | Run query | Submit query

Database | Result

order_id	shipped_date	shipping_rank
10249	1996-07-10	1
10252	1996-07-11	2
10250	1996-07-12	3
10255	1996-07-15	4
10251	1996-07-15	4

Description | Resources

(set of rows). OVER clause is used with window functions to define that window. OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS new_column
FROM table_name;
```

Questions 3/9

3. Create order_rank for all the orders based on the order data. The latest order should get the first rank Also, observe if more than one order_Id have gotten the same rank or not.

   Expected Output: Order_id, order_date and order_rank.

Query

```
select
 order_id,
 order_date,
 rank() over(order by order_date desc) as order_rank
from orders
order by 2 desc
```

Reset | Run query | Submit query

Database | Result

order_id	order_date	order_rank
11074	1998-05-06	1
11077	1998-05-06	1
11076	1998-05-06	1
11075	1998-05-06	1
11071	1998-05-05	5

Description | Resources

Window functions apply aggregate and ranking functions over a particular window (set of rows). OVER clause is used with window functions to define that window. OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS
new_column
FROM table_name;
```

Questions 4/9

4. Company wants to give $5 cashback to every customer on the first order. Identify the order_ids and customer_ids which are eligible for getting the cashback.

   Expected Output: Customer_id and first_order_id. The output should be sorted by customer_id first then order_id

Query

```
with ranked_orders as(select
customer_id,
order_id,
order_date,
row_number() over(partition by customer_id order by order_date asc,order_id asc) as order_rank
from orders)
select
customer_id,
order_id as first_order_id
from ranked_orders
where order_rank =1
order by 1,2 asc
```

Reset | Run query | Submit query

Database | Result

customer_id	first_order_id
ALFKI	10643
ANATR	10308
ANTON	10365
AROUT	10355
BERGS	10278

Description | Resources

OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS
new_column
FROM table_name;
```

Questions 5/9

5. To increase the sale of a new employee, bonus is given on the first sale of the employee. Identify the first order_id for every employee id using order_date in the orders table.

   Expected Output - Employee_id and First order_id in the same sequence.

Query

```
with ranked_orders as (select
 employee_id,
 order_id,
 order_date,
 row_number() over(partition by employee_id order by order_date asc, order_id asc) as order_rank
from orders)
select
 employee_id,
 order_id as first_order_id
from ranked_orders
where order_rank =1
order by 1,2 asc
```

Reset | Run query | Submit query

Database | Result

employee_id	first_order_id
1	10258
2	10265
3	10251
4	10250
5	10248

Description | Resources

```
new_column
FROM table_name;
```

Questions 6/9

6. Identify first order_id (using order_date) for every month.

   Expected Output: Year_month in 'YYYY-MM' format and first order_id. Final output should be sorted in ascending order of Year_month

```
order_rank
FROM year_month_orders)

SELECT
Year_month,
order_id
FROM Rank_application
WHERE order_rank = 1
```

Query

```
with year_month_orders as(select
 to_char(order_date,'YYYY-MM') as Year_month,
 order_id,
 order_date
from orders),
rank_applications as(select
 *,
 rank() over(partition by Year_month order by order_date asc) as order_rank
from year_month_orders)
select
 Year_month,
 order_id as first_order_id
from rank_applications
where order_rank =1
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

year_month	first_order_id
1996-07	10248
1996-08	10270
1996-08	10271
1996-09	10295
1996-10	10318

Description | Resources

```
FROM table_name;
```

Questions 7/9

7. Identify highest selling (quantity sold) product_id in every category_id

   Expected Output: Category_id, Highest_selling_product_id and quantity_sold. The output should be sorted by category_id in ascending order

```
FROM product_category_wise_Quantity_sold
)
SELECT
category_id,
product_id,
quantity_sold
FROM rank_data WHERE product_rank = 1

ORDER BY 1 ASC
```

Query

```
with product_category_wise_qty_sold as(select
 p.category_id,
 o.product_id,
 sum(quantity) as quantity_sold
from products p
inner join order_details o
on p.product_id = o.product_id
group by 1,2),
rank_data as(select
 *,
 rank() over(partition by category_id order by quantity_sold desc) as product_rank
from product_category_wise_qty_sold)
select
 category_id,
 product_id as highest_selling_product_id,
 quantity_sold
from rank_data
where product_rank =1
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

category_id	highest_selling_product_id	quantity_sold
1	75	1155
2	77	791
3	16	1158

Description | Resources

(set of rows). OVER clause is used with window functions to define that window. OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS new_column
FROM table_name;
```

Questions 8/9

8. Identify least selling (quantity sold) product_id in every category_id

   Expected Output: Category_id, Highest_selling_product_id, and quantity_sold. The output should be sorted by category_id in ascending order

Query

```
with product_category_wise_qty_sold as(
select
 p.category_id,
 o.product_id,
 sum(o.quantity) as quantity_sold
from products p
inner join order_details o
on p.product_id = o.product_id
group by 1,2),
rank_data as(select
 *,
 rank() over(partition by category_id order by quantity_sold asc) as product_rank
from product_category_wise_qty_sold)
select
 category_id,
 product_id as highest_selling_product_id,
 quantity_sold
from rank_data
where product_rank =1
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

category_id	highest_selling_product_id	quantity_sold
1	67	184
2	15	122
3	48	138

Description | Resources

OVER clause does two things :

- Partitions rows to form set of rows. (PARTITION BY clause is used)
- Orders rows within those partitions into a particular order. (ORDER BY clause is used)

Syntax

```
SELECT coulmn_name1,
 window_function(cloumn_name2)
 OVER([PARTITION BY column_name1] [ORDER BY column_name3]) AS new_column
FROM table_name;
```

Questions 9/9

9. Identify highest selling (quantity sold) product_id in every category_name

   Expected Output: Category_name, Highest_selling_product_id and quantity_sold. The output should be sorted by category_id in ascending order

Query

```
 c.category_name,
 p.product_id,
 sum(o.quantity) as quantity_sold
from categories c
inner join products p
on p.category_id = c.category_id
inner join order_details o
on p.product_id = o.product_id
group by 1,2),
rank_data as(select
 *,
 rank() over(partition by category_name order by quantity_sold desc) as product_rank
from category_product_wise_qty_sold)
select
 category_name,
 product_id as highest_selling_product_id,
 quantity_sold
from rank_data
where product_rank =1
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

category_name	highest_selling_product_id	quantity_sold
Beverages	75	1155
Condiments	77	791
Confections	16	1158

Description

Questions 1/5

1. Each Employee is assigned one territory and is responsible for the Customers from that territory. There may be multiple employees assigned to the same territory. Write a query to get the Employees who are responsible for the maximum number of Customers.

Expected output:

`empl_id, total_customers`, The output should be sorted by `empl_id` in ascending order.

Hint

You need to join 2 tables:

```
map_employee_territory,
map_customer_territory
```

Query

```
select
 empl_id,
 total_customers
from (select
 empl_id,
 count(cust_id) as total_customers,
 rank()over(order by count(cust_id)desc)rank
from map_employee_territory e
inner join map_customer_territory ct
 on e.territory_id = ct.territory_id
 group by 1) a
where rank =1
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

empl_id	total_customers
E849	4
E850	4
E851	4
E856	4

Description

Questions 2/5

2. Write a query to find the Market Share at the Product Brand level for each Territory, for Time Period Q4-2021. Market Share is the number of Products of a certain Product Brand brand sold in a territory, divided by the total number of Products sold in this Territory. Only include these Product Brands that had at least one sale in a given territory.

Expected output:

`territory_id, prod_brand, market_share`

, The output should be sorted by `territory_id, prod_brand` in ascending order

Hint

You need to join 3 tables:

Query

```
with sales_per_brand_territory as(select
 prod_brand,
 territory_id,
 count(*) as n_sales
from fct_customer_sales s
inner join map_customer_territory t
on s.cust_id = t.cust_id
inner join dim_product p
on s.prod_sku_id = p.prod_sku_id
where extract(year from order_date) =2021 and extract(quarter from order_date)=4
group by 1,2)
select
 territory_id,
 prod_brand,
 n_sales/sum(n_sales)over(partition by territory_id)*100 as market_share
from sales_per_brand_territory
order by 1,2
```

Reset | Run query | Submit query

Database | Result

territory_id	prod_brand	market_share
T1	Apple	33.33333333333333333300
T1	JBL	16.66666666666666666700
T1	Samsung	50.00000000000000000000
T2	Apple	25.00000000000000000000

Showing 19 of 19 records

Description

Questions 3/5

3. Write a query to return Territory and corresponding Sales Growth. Compare growth between periods Q4-2021 vs Q3-2021. If Territory (say T123) has Sales worth $100 in Q3-2021 and Sales worth $110 in Q4-2021, then the Sales Growth will be 10% [i.e. = ((110 - 100)/100) * 100] Output the ID of the Territory and the Sales Growth. Only output these territories that had any sales in both quarters.

Expected output:

`territory_id, sales_growth`, The output should be sorted by `territory_id` in ascending order. Round off the `sales_growth` to 2 decimal points.

Hint

Join 2 tables:

Query

```
with Quarter_territory_wise_sales as(select
 t.territory_id,
 extract(quarter from order_date) as quarter,
 sum(order_value) as total_order_value
from fct_customer_sales c
inner join map_customer_territory t
on c.cust_id = t.cust_id
where extract(year from order_date) =2021 and extract(quarter from order_date) in (3,4)
group by 1,2)
select
 a.territory_id,
 round(cast((b.total_order_value-a.total_order_value) as decimal)/cast((a.total_order_value) as decimal)*100,2) as sales_growth
from Quarter_territory_wise_sales a
inner join Quarter_territory_wise_sales b
on a.territory_id = b.territory_id
and a.quarter < b.quarter
```

Reset | Run query | Submit query

Database | Result

territory_id	sales_growth
T1	4.44
T3	67.12
T4	17.36
T5	6.07

Description

- string functions

Questions 4/5

4. Find the top selling brand in each territory.

Expected output:
`territory_id, prod_brand`. The output should be sorted by `territory_id` in ascending order

Hint

Join 3 tables:
`map_customer_territory, fct_customer_sales, dim_product`
and use the `row_number()` function to get the desired output.

Query

```
with brand_wise_territory_sales as(select
t.territory_id,
p.prod_brand,
count(s.order_value) as total_sales
from fct_customer_sales s
inner join map_customer_territory t
on t.cust_id = s.cust_id
inner join dim_product p
on p.prod_sku_id = s.prod_sku_id
group by 1,2),
ranked_brands as(select
territory_id,
prod_brand,
row_number() over(partition by territory_id order by total_sales desc) as sales_rank
from brand_wise_territory_sales)
select
territory_id,
prod_brand
from ranked_brands
where sales_rank =1
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

territory_id	prod_brand
T1	Samsung
T2	Samsung

Description

Statements I

Some of the advance SQL statements that we will be covering here are:

- rank functions
- date functions
- windows functions
- string functions

Questions 5/5

5. Retrieve the month and total number of brands for each month where the total number of brands is greater than 10.

Expected output:
`month, total_brands`. The output should be sorted by month in ascending order.

Query

```
select
 extract(month from c.order_date) as month,
 count(p.prod_sku_id) as total_brands
from fct_customer_sales c
inner join dim_product p
on p.prod_sku_id = c.prod_sku_id
group by 1
having count(p.prod_sku_id) >10
order by 1 asc
```

Reset | Run query | Submit query

Database | Result

month	total_brands
7	11
8	13
10	14
11	15
12	13

Description

Practice Problems: JOIN with aggregation II

Perform aggregation on JOIN tables

Questions 1/4

1. Calculate order_id wise total payment paid by every customer_id till date. Total payment = unit_price x quantity x (1-discount). Name the newly created column as total_payment else the query is not going to be accepted

Expected Output: Customer_id, order_id, and total_payment. The output should be in decreasing order of order_id

Query

```
select
 o.customer_id,
 o.order_id,
 sum(od.unit_price*od.quantity*(1-od.discount)) as total_payment
from orders o
inner join order_details od
on o.order_id = od.order_id
group by 1,2
order by 2 desc
```

Reset | Run query | Submit query

Database | Result

customer_id	order_id	total_payment
RATTC	11077	1255.7204990328737
BONAP	11076	792.7499985694885
RICSU	11075	498.0999965071678
SIMOB	11074	232.08500996507703
PERIC	11073	300
ERNSH	11072	5217.999984741211

Showing 500 of 500 records

Description

Practice Problems: JOIN with aggregation II

Perform aggregation on JOIN tables

Questions 2/4

2. Calculate total payment paid by every customer_id till date. Total payment = unit_price x quantity x (1-discount). Name the newly created column as total_payment else the query is not going to be accepted

   Expected Output: Customer_id and total_payment. The output should be in decreasing order of total_payment

Query

```sql
select
 o.customer_id,
 sum(unit_price*quantity*(1-discount)) as total_payment
from orders o
inner join order_details od
on o.order_id = od.order_id
group by 1
order by 2 desc
```

Reset | Run query | Submit query

Database | Result

customer_id	total_payment
QUICK	110277.30503039382
ERNSH	104874.97814367746
SAVEA	104361.94954039395
RATTC	51097.80082826822
HUNGO	49979.90508149549

Description

Practice Problems: JOIN with aggregation II

Perform aggregation on JOIN tables

Questions 3/4

3. In previous table, we had customer_id wise total payment. But its very difficult to identify the customer from customer_id. Add the customer's contact name in the previous table as well.

   Expected Out: customer_id, contact_name and total_payment. The final output should be sorted by total_payment column

Hint

SELECT

Query

```sql
select
 o.customer_id,
 c.contact_name,
 sum(unit_price*quantity*(1-discount)) as total_payment
from orders o
inner join customers c
on o.customer_id = c.customer_id
inner join order_details od
on o.order_id = od.order_id
group by 1,2
order by 3 desc
```

Reset | Run query | Submit query

Database | Result

customer_id	contact_name	total_payment
QUICK	Horst Kloss	110277.30503039382
ERNSH	Roland Mendel	104874.97814367746
SAVEA	Jose Pavarotti	104361.94954039395
RATTC	Paula Wilson	51097.80082826822
HUNGO	Patricia McKenna	49979.90508149549

Description

Practice Problems: JOIN with aggregation II

Perform aggregation on JOIN tables

Questions 4/4

4. Calculate total quantity ordered against every customer's contact_name.

   Expected Output: contact_name and total quantity ordered. Sort the output by quantity in descending order

Query

```sql
select
 c.contact_name,
 sum(quantity) as total_quantity
from customers c
inner join orders o
on c.customer_id = o.customer_id
inner join order_details od
on o.order_id = od.order_id
group by 1
order by 2 desc
```

Reset | Run query | Submit query

Database | Result

contact_name	total_quantity
Jose Pavarotti	4958
Roland Mendel	4543
Horst Kloss	3961
Patricia McKenna	1684
Peter Franken	1525
Paula Wilson	1383

Description

Practice Problems: JOINS III

SQL `JOINS` are used to retrieve data from multiple tables. A SQL `JOIN` is performed whenever two or more tables are listed in a SQL statement.

There are 4 different types of SQL joins:

`SQL INNER JOIN` (sometimes called simple `JOIN`)
`SQL LEFT OUTER JOIN` (sometimes called `LEFT JOIN`)
`SQL RIGHT OUTER JOIN` (sometimes called `RIGHT JOIN`)
`SQL FULL OUTER JOIN` (sometimes called `FULL JOIN`)

Questions 1/5

1. Find all the pizza ids for every month where the order count is more than

Query

```sql
select
 extract(month from o.date) as month,
 od.pizza_id,
 count(od.order_details_id) as total_orders
from orders o
inner join order_details od
on o.order_id = od.order_id
inner join pizzas p
on p.pizza_id = od.pizza_id
group by 1,2
having count(od.order_details_id) >100
order by 1 asc, 2 asc
```

Reset | Run query | Submit query

Database | Result

month	pizza_id	total_orders
1	big_meat_s	146
1	five_cheese_l	134
1	thai_ckn_l	117
2	big_meat_s	144
2	five_cheese_l	112
2	four_cheese_l	111

Description

`SQL RIGHT OUTER JOIN` (sometimes called `RIGHT JOIN`)
`SQL FULL OUTER JOIN` (sometimes called `FULL JOIN`)

Questions 2/5

2. Retrieve the total number of pizzas for each category.

   Expected output: `category, total_pizzas`. The output should be sorted by `category` in ascending order.

Hint

You need to join 3 tables: `pizza_types, pizzas, order_details` to get the desired output

Expected Query:

Query

```sql
select
 t.category,
 sum(od.quantity) as total_pizzas
from pizza_types t
inner join pizzas p
on t.pizza_type_id = p.pizza_type_id
inner join order_details od
on p.pizza_id = od.pizza_id
group by 1
order by 1
```

Reset | Run query | Submit query

Database | Result

PIZZA_TYPES | ORDERS | ORDER_DETAILS | PIZZAS

pizza_type_id	name	category	ingredients
bbq_ckn	The Barbecue Chicken Pizza	Chicken	Barbecued Chicken, Red Peppers, Green Peppe
cali_ckn	The California Chicken Pizza	Chicken	Chicken, Artichoke, Spinach, Garlic, Jalapeno F
ckn_alfredo	The Chicken Alfredo Pizza	Chicken	Chicken, Red Onions, Red Peppers, Mushroom
ckn_pesto	The Chicken Pesto Pizza	Chicken	Chicken, Tomatoes, Red Peppers, Spinach, Gar

Description

`RIGHT JOIN`
`SQL FULL OUTER JOIN` (sometimes called `FULL JOIN`)

Questions 4/5

4. Find all the pizza types that are ordered more than 2000 times.

   Expected output: `pizza_type_id`

Expected Query:

```
Select pizza_type_id
from order_details od
join pizzas p
on od.pizza_id=p.pizza_id
group by 1
having count(pizza_type_id)>2000
order by 1
```

Query

```sql
select
 p.pizza_type_id
 from pizzas p
inner join order_details od
on p.pizza_id = od.pizza_id
group by 1
having count(od.order_id) >2000
order by 1
```

Reset | Run query | Submit query

Database | Result

pizza_type_id
bbq_ckn
cali_ckn
classic_dlx
hawaiian
pepperoni
thai_ckn

Description

There are 4 different types of SQL joins:

`SQL INNER JOIN` (sometimes called simple `JOIN`)

`SQL LEFT OUTER JOIN` (sometimes called `LEFT JOIN`)

`SQL RIGHT OUTER JOIN` (sometimes called `RIGHT JOIN`)

`SQL FULL OUTER JOIN` (sometimes called `FULL JOIN`)

Questions 5/5

5. Find total number of orders placed in every month and pizza name.

   Expected output: `month, name, total_orders`, The output should be sorted by month and `name` in ascending order.

Query

```
select
 extract(month from o.date) as month,
 t.name,
 count(od.order_id) as total_orders
from pizza_types t
inner join pizzas p
on t.pizza_type_id = p.pizza_type_id
inner join order_details od
on od.pizza_id = p.pizza_id
inner join orders o
on o.order_id = od.order_id
group by 1,2
order by month asc, name asc
```

Reset | Run query | Submit query

Database | Result

month	name	total_orders
1	The Barbecue Chicken Pizza	206
1	The Big Meat Pizza	146
1	The Brie Carre Pizza	34
1	The Calabrese Pizza	66
1	The California Chicken Pizza	197
1	The Chicken Alfredo Pizza	83

Description

Practice Problems: LEFT JOIN IV

The `LEFT JOIN` is a clause of the `SELECT` statement. It allows you to query data from multiple tables and returns all rows from the left table and the matching rows from the right table. If no matching rows are found in the right table, `NULL` are used.

Questions 2/2

2. Find the details of each book regardless of whether the book was sold.

   Expected output:

   `book_id, book_title, order_details_id, order_id, quantity, unit_price`

   , you may have duplicate rows in your results due to a customer ordering several of the same items. Sort records based on the book's name and details in ascending order.

Query

```
select
 b.book_id,
 b.book_title,
 o.order_details_id,
 o.order_id,
 o.quantity,
 b.unit_price
from amazon_books b
left join amazon_books_order_details o
on b.book_id = o.book_id
order by b.book_title asc, o.order_details_id asc
```

Reset | Run query | Submit query

Database | Result

book_id	book_title	order_details_id	order_id	quantity	unit_price
B007	Animal Farm	OD115	O1010	2	40
B009	Fahrenheit 451	OD102	O1001	1	50
B009	Fahrenheit 451	OD130	O1020	1	50
B010	Gone with the Wind	OD124	O1015	2	25
B019	Jane Eyre	OD105	O1002	2	60

Description

No Hint Problem : Windows Functions 1

Questions 1/2

1. Fetch the top 3 customers who placed the highest orders in each store's city, along with the total number of orders placed. Assign the same rank to the same value, and the rank should be skipped.
   Expected Output: city, customer_full_name(First name space last name), total_orders_placed in the same sequence.
   Sort city name in ascending order then customer name in ascending order and then total orders placed in descending order.

Query

```sql
with customerorders as(select
 s.city,
 concat(c.first_name, ' ', c.last_name) as customer_full_name,
 count(o.order_id) as total_orders_placed,
 rank() over(partition by s.city order by count(o.order_id) desc) as customer_rnk
from customers c
join orders o
on c.customer_id = o.customer_id
join stores s
on o.store_id = s.store_id
group by 1,2)
select
 city,
 customer_full_name,
 total_orders_placed
from customerorders
where customer_rnk <=3
order by 1 asc, 2 asc, 3 desc
```

Reset | Run query | Submit query

Database | Result

city	customer_full_name	total_orders_placed
Baldwin	Caren Stephens	3
Baldwin	Daryl Spence	3
Baldwin	Debra Burks	3
Baldwin	Deshawn Mendoza	3
Baldwin	Emmitt Sanchez	3

Description

No Hint Problem : Windows Functions 1

Questions 2/2

2. Find the top 5 staff members who generated the highest revenue in each category and their total revenue. Consider revenue = (quantity * list_price * (1-discount)).
   Expected Output: category_id, staff_id, total_revenue, rank in the same sequence.
   Sort the total_revenue in descending order, then category_id in ascending order

Query

```sql
with staff_with_highrevenue as(select
 p.category_id,
 s.staff_id,
 sum(od.quantity*od.list_price*(1-od.discount)) as total_revenue
from products p
inner join order_items od
on p.product_id = od.product_id
inner join orders o
on od.order_id = o.order_id
inner join staffs s
on s.staff_id = o.staff_id
group by 1,2),
ranked_revenue as(select
 * ,
rank() over(partition by category_id order by total_revenue desc) as rnk
from staff_with_highrevenue)
select
 category_id,
 staff_id,
 total_revenue,
 rnk
from ranked_revenue
where rnk <=5
order by 3 desc,1 asc
```

Reset | Run query | Submit query

Database | Result

category_id	staff_id	total_revenue	rnk
6	6	930168.3645	1
6	7	906223.8619	2
7	6	576487.0894	1

Showing 35 of 35 records

Description

No Hint Problem : Windows functions 2

Questions

1. Identify the top 3 earning staff each month, ranking them based on the total revenue they generated.

   Consider revenue = (quantity * list_price * (1-discount)).
   Expected Output: Staff_id, Month, Total_revenue, Rank in the same sequence.
   Sort the Total_revenue in descending order and then Month in ascending order.

Query

```sql
with revenue_generated_staff as(select
 s.staff_id,
 extract(month from o.order_date) as month,
 sum(od.quantity*od.list_price*(1-od.discount)) as total_revenue
from staffs s
inner join orders o
on s.staff_id = o.staff_id
inner join order_items od
on o.order_id = od.order_id
group by 1,2),
ranked_orders as(select
 *,
 rank() over(partition by month order by total_revenue desc) as ranked
from revenue_generated_staff)
select
 staff_id,
 month,
 total_revenue,
 ranked
from ranked_orders
where ranked <=3
order by 3 desc, 2 asc
```

Reset | Run query | Submit query

Database | Result

staff_id	month	total_revenue	ranked
7	4	450852.5076	1
6	1	343156.1251	1
7	3	310723.9035	1
6	3	302154.4874	2

Showing 36 of 36 records

Description

No Hint Problem : Windows functions 3

Questions 1/2

1. Identify restaurants with the most fluctuating (overall ratings) ratings by calculating the range between the highest and lowest overall ratings and rank these from most to least fluctuating (with rank 1 being the highest).
   Consider rating_range = MAX(overall_rating) - MIN(overall_rating).
   Expected Output: restaurant_id, Rating_range, Fluctuation_rank in the same sequence
   Sort the Rating_range in descending order and then restaurant_id in ascending order.

Hint

No hint will be provided.

Query

```sql
with rating_range as(select
 r.restaurant_id,
 (max(ra.overall_rating)-min(ra.overall_rating)) as rating_range
from restaurants r
inner join ratings ra
on r.restaurant_id = ra.restaurant_id
group by 1),
ranked_ratings as(select
 *,
 rank() over(order by rating_range desc) as fluctuation_rank
from rating_range)
select
 restaurant_id,
 rating_range,
 fluctuation_rank
from ranked_ratings
order by 2 desc,1 asc
```

Reset | Run query | Submit query

Database | Result

restaurant_id	rating_range	fluctuation_rank
132560	4	1
132561	4	1
132564	4	1
132572	4	1
132583	4	1
132584	4	1

Description

SQL No Hint Problem : SQL Conditional Functions 2

Questions 1/2

1. Fetch customer spending per year, rank them based on their spending, and categorise their spending as 'High', 'Medium', or 'Low'. Consider spending = (quantity * list_price * (1-discount)). Assign the same rank to the same value, and the rank should not be skipped.
   High: Spending rank should be Between 1 and 10.
   Medium: Spending rank should be Between 11 and 50
   Low: Spending rank should be greater than 50. (for ex: 51,52.....)
   Expected Output: customer_id, Year, Total_spent, Spending_rank, Spending_category in the same sequence.
   Sort the total spent in descending order.

Query

```sql
with cte1 as(select
 o.customer_id,
 extract(year from o.order_date) as year,
 sum(oi.quantity*oi.list_price*(1-oi.discount)) as total_spent
from orders o
join order_items oi
on o.order_id = oi.order_id
group by 1,2),
cte2 as(select
 customer_id,
 year,
 total_spent,
 dense_rank() over(partition by year order by total_spent desc) as spending_rank
from cte1)
select
 customer_id,
 year,
 total_spent,
 spending_rank,
 case
  when spending_rank between 1 and 10 then 'High'
  when spending_rank between 11 and 50 then 'Medium'
  else 'Low'
 end as spending_category
from cte2
order by total_spent desc
```

Reset | Run query | Submit query

Database | Result

Description

SQL No Hint Problem : SQL Conditional Functions 6

Questions 1/3

1. Identify products in each price range whose sales peak in the fourth quarter of 2020 and rank them by total sales within their price range (with rank 1 being the highest).
   Consider total sales = orderquantity.
   Consider price_range as follows:
   High: productprice greater than 30.
   Medium: productprice between 20 and 30.
   Low: productprice less than 20.
   Assign the same rank to the same value, and the rank should be skipped.
   Expected Output: productname, Total_sales, Price_range, Rank in the same sequence.
   Sort the Total_sales in descending order then productname in ascending order.

Query

```sql
with quarter4_sales as(select
 p.productname,
 p.productprice,
 sum(s.orderquantity) as total_sales
from products p
inner join sales s
on p.product_id = s.product_id
where s.orderdate >= '2020-10-01' and orderdate < '2021-01-01'
group by 1,2),
priced as(select
 productname,
 total_sales,
 case
  when productprice > 30 then 'High'
  when productprice between 20 and 30 then 'Medium'
  else 'Low'
  end as price_range
from quarter4_sales),
ranked as(select
 productname,
 total_sales,
 price_range,
 rank() over(partition by price_range order by total_sales desc) as rank_in_range
from priced)
select
 productname,
 total_sales,
 price_range,
 rank_in_range
from ranked
order by 2 desc,1 asc
```

Reset | Run query | Submit query

Database | Result

productname	total_sales	price_range	rank_in_range
Road-250 Black, 52	56	High	1
Mountain-200 Black, 42	47	High	2
Road-250 Red, 58	47	High	2

Description

No Hint Problem : Windows Functions 5

Questions 1/2

1. Identify the cuisine type with the most fluctuating overall ratings across all restaurants offering that cuisine and rank them (with rank 1 being the highest). Use VAR_SAMP to get the rating variance. Assign the same rank to the same value, and the rank should be skipped.
Expected Output: cuisine_id, Rating_variance (rounded to two decimal points), variance_rank in the same sequence.
Sort the rating_variance in descending order, then cuisine_id in ascending order.

Query

```
with cuisine_var as(select
 rc.cuisine_id,
 round(var_samp(ra.overall_rating),2) as rating_variance
from ratings ra
inner join restaurant_cuisines rc
on ra.restaurant_id = rc.restaurant_id
group by 1),
ranked as(select
 cuisine_id,
 rating_variance,
 rank() over(order by rating_variance desc) as variance_rank
from cuisine_var)
select
 cuisine_id,
 rating_variance,
 variance_rank
from ranked
order by 2 desc,1 asc
```

Reset | Run query | Submit query

Database | Result

cuisine_id	rating_variance	variance_rank
76	2.40	1
26	2.34	2
74	2.33	3
18	2.26	4
31	2.24	5
83	2.21	6

Description

No Hint Problem : Windows Functions 6

Questions 1/2

1. Rank customers by their annual income within each marital status (with 1 being the highest), displaying customer ID, name, marital status, and income rank. Assign the same rank to the same value, and the rank should be skipped.
Expected Output: customer_id, firstname, maritalstatus, annualincome, income_rank in the same sequence.
Sort the annualincome in descending order then firstname in ascending order.
Show only the top 50 rows.

Query

```
with cust_annual_inc as(select
customer_id,
firstname,
maritalstatus,
annualincome
from customers),
ranked as(select
 customer_id,
 firstname,
 maritalstatus,
 annualincome,
 rank()over( partition by maritalstatus order by annualincome desc) as income_rank
from cust_annual_inc)
select
customer_id,
firstname,
maritalstatus,
annualincome,
income_rank
from ranked
order by 4 desc,2 asc
limit 50
```

Reset | Run query | Submit query

Database | Result

customer_id	firstname	maritalstatus	annualincome	income_rank
27663	AARON	S	170000	1
14251	ALEX	M	170000	1
11244	ALEXIS	S	170000	1
28914	ALICIA	M	170000	1

Showing 50 of 50 records

Description

No Hint Problem : Windows Functions 8

Questions 1/2

1. Determine the first sale date for each product and rank these dates within their respective subcategories. Assign the same rank to the same value, and the rank should be skipped.
Expected Output:
productname, Subcategoryname, First_sale_date, Rank in the same sequence.
Sort the First_sale_date in ascending order then Rank in ascending order and then productname in ascending order.

Query

```sql
with first_sale as(select
 p.productname,
 sub.subcategoryname,
 min(s.orderdate) as first_sale_date
from products p
inner join sales s
on p.product_id = s.product_id
inner join subcategories sub
on sub.subcategory_id = p.subcategory_id
group by 1,2)
select
 productname,
 subcategoryname,
 first_sale_date,
 rank() over(partition by subcategoryname order by first_sale_date) as rank
from first_sale
order by 3 asc,4 asc,1 asc
```

Reset | Run query | Submit query

Database | Result

productname	subcategoryname	first_sale_date	rank
Road-550-W Yellow, 42	Road Bikes	2020-07-01	18
Road-250 Black, 52	Road Bikes	2020-07-02	21
Road-550-W Yellow, 38	Road Bikes	2020-07-02	21
Road-250 Black, 58	Road Bikes	2020-07-03	23
Road-250 Red, 44	Road Bikes	2020-07-03	23
Road-250 Red, 48	Road Bikes	2020-07-03	23

Description

No Hint Problem : Windows Functions 10

Questions 1/2

1. Rank the years based on their total sales revenue, considering the sales generated from orders yearly.
Consider revenue = (quantity * list_price * (1-discount)).
Expected Output:
Year,Total_sales, year_rank in the same sequence
Sort the Total_sales in descending order and then year in ascending order.

Query

```sql
with yearly_orders as(select
 extract(year from o.order_date) as year,
 sum(oi.quantity*oi.list_price*(1-oi.discount)) as total_sales
from orders o
inner join order_items oi
on o.order_id = oi.order_id
group by 1),
ranked as(select
 year,
 total_sales,
 rank() over(order by total_sales desc) as year_rank
from yearly_orders)
select
*
from ranked
order by total_sales desc, year asc
```

Reset | Run query | Submit query

Database | Result

year	total_sales	year_rank
2017	3447208.2425	1
2016	2427378.5276	2
2018	1814529.7875	3

Description

No Hint Problem : Windows Functions 10

Questions 2/2

2. Determine the top 5 best-selling products during weekdays and assign rank according to the sales. Assign the same rank to the same value, and the rank should not be skipped.
Consider sales = quantity * list_price, and weekdays= (Monday to Friday). To find the day of the week, use DOW with the extract function.
Expected Output: product_id, sales, rank in the same sequence.
Sort the sales in descending order.

Query

```
with product_sales as(select
 oi.product_id,
 sum(oi.quantity*oi.list_price) as sales
from orders o
inner join order_items oi
on o.order_id = oi.order_id
where extract(DOW from o.order_date) between 1 and 5
group by 1),
ranked as(select
 product_id,
 sales,
 dense_rank() over(order by sales desc) as rank
from product_sales)
select
 product_id,
 sales,
 rank
from ranked
where rank <=5
order by 2 desc
```

Reset | Run query | Submit query

Database | Result

product_id	sales	rank
7	439998.90	1
9	320998.93	2
4	263899.09	3
56	192499.65	4
11	169779.99	5

Description

No Hint Problem : Windows Functions 11

Questions 2/2

2. Retrieve the day of the week with the highest total revenue in a single day along with the month and year. Consider revenue = (quantity * list_price * (1-discount)).
(Day_of_the_week should be in Sunday, Monday format).
Show only the top row.
The month should be in 1.2.3.... format.
Expected Output:
day_of_the_week, month, year, Total_revenue, rank in the same sequence.
Sort the Total_revenue in the descending order.

Query

```
with total_revenue as(select
 to_char(o.order_date, 'Day') as day_of_the_week,
 extract(month from o.order_date) as month,
 extract(year from o.order_date) as year,
 sum(oi.quantity*oi.list_price*(1-oi.discount)) as total_revenue
from orders o
inner join order_items oi
on o.order_id = oi.order_id
group by 1,2,3),
ranked as(select
 day_of_the_week,
 month,
 year,
 total_revenue,
 rank() over(order by total_revenue desc) as rank
from total_revenue)
select
 day_of_the_week,
 month,
 year,
 total_revenue,
 rank
from ranked
where rank =1
order by total_revenue desc
```

Reset | Run query | Submit query

Database | Result

day_of_the_week	month	year	total_revenue	rank
Sunday	4	2018	147227.6130	1

Description

SQL No Hint Problem : SQL Conditional Functions 1

Questions 1/2

1. Retrieve the total revenue for each month along with the corresponding season, and rank them based on total revenue.
   The definition of the season is mentioned below:
   Spring: March, April, and May
   Summer: June, July, and August
   RainFall: September, October, and November
   Winter: December, January, and February
   The month should be in 1.2.3.... format.
   Consider revenue = (quantity * list_price * (1-discount)).
   Expected Output: month, Total_sales, Season, sales_rank in the same sequence.
   Sort the Total_sales in descending order then season in ascending order.

Query

```sql
with total_rev as(select
 extract(month from o.order_date) as month,
 sum(oi.quantity*oi.list_price*(1-oi.discount)) as total_sales
from orders o
inner join order_items oi
on o.order_id = oi.order_id
group by 1),
season_case as(select
 month,
 total_sales,
 case
 when month in(3,4,5) then 'Spring'
 when month in(6,7,8) then 'Summer'
 when month in(9,10,11) then 'RainFall'
 when month in(12,1,2) then 'Winter'
end as season,
 rank() over(order by total_sales desc) as sales_rank
from total_rev)
select
 month,
 total_sales,
 season,
 sales_rank
from season_case
order by 2 desc, 3 asc
```

Reset | Run query | Submit query

Database | Result

month	total_sales	season	sales_rank
4	1212356.8247	Spring	1
1	882193.0074	Winter	2

Description

Questions 2/2

2. Determine the monthly sales ranking of products while categorising months into 'High Season', 'Medium Season', or 'Low Season' based on their sales volume.
   Consider sales = quantity * list_price.
   High Season: June, July, August.
   Medium Season: September, October, November.
   Low Season: January, February, March, April, May, December.
   Expected Output: month,total_sales, season,sales_rank in the same sequence.
   Sort the total sales in descending order.

Query

```sql
with monthly_sales as(select
 extract(month from o.order_date) as month,
 sum(oi.quantity*oi.list_price) as total_sales
from orders o
inner join order_items oi
on o.order_id = oi.order_id
group by 1),
case_season as(select
 month,
 total_sales,
 case
 when month in(6,7,8) then 'High Season'
 when month in(9,10,11) then 'Medium Season'
 else 'Low Season'
 end as season
from monthly_sales),
ranked as(select
 month,
 total_sales,
 season,
 rank() over(order by total_sales desc) as sales_rank
from case_season)
select
 month,
 total_sales,
 season,
 sales_rank
from ranked
order by 2 desc
```

Reset | Run query | Submit query

Database | Result

month	total_sales	season	sales_rank